## Distributed Resource management

A distributed file system is a resource management component of a distributed operating system. It implements a common file system that can be shared by all the autonomous computers in the system. Two important goals of distributed file systems follows

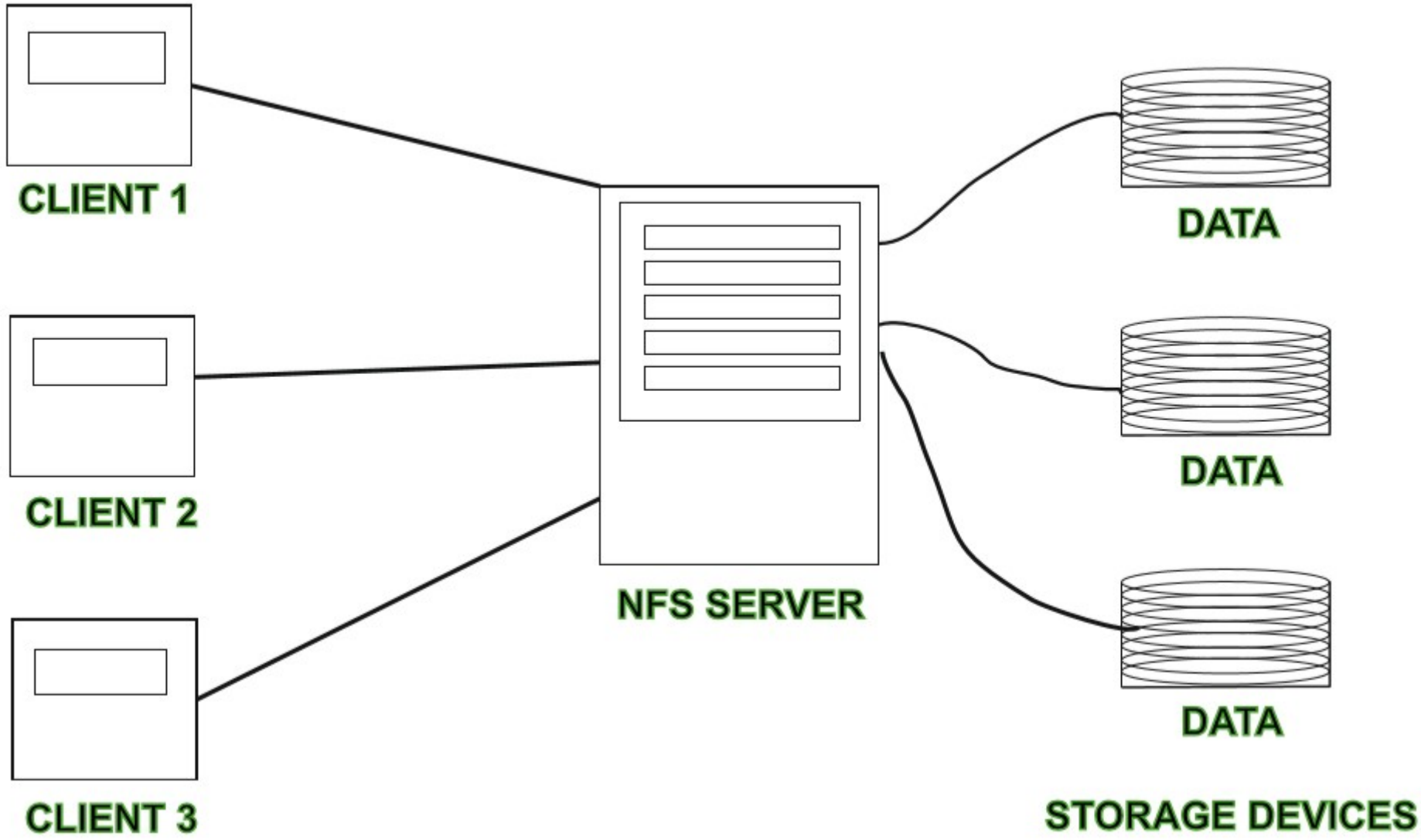
1. Network transparency:- The primary goal of distributed file system is to provide the same functional capabilities to accesses files distributed over a network as the file system of a time sharing mainframe system does to access files residing at one location. Users do not have to be aware of the location of files to access them.

2. High availability :- Users should have the same easy access to files, irrespective of their physical location. System failures or regularly scheduled activities such as backups or maintenance should not result in the unavailability of files.

The two most important services present in distributed file system are the name server and cache manager. A name server is a process that maps names specified by clients to stored objects such as files and directories.

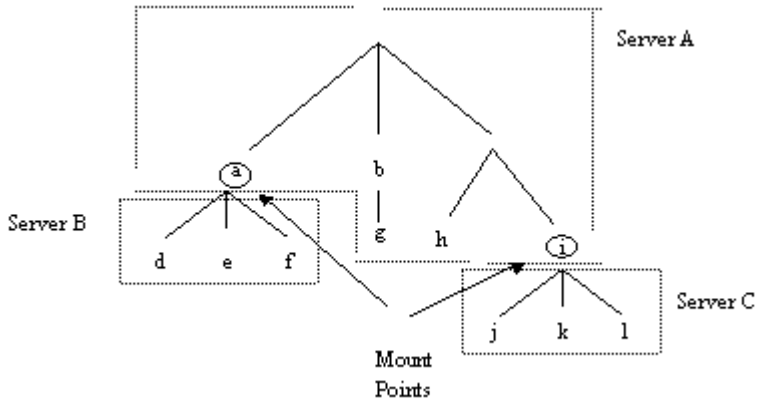
A cache manager is a process that implements file caching. I file caching a copy of data stored at a remote file server is brought to the client's  machine when referenced by the client. Cache managers can be present at both clients and file servers.

# Architecture of Distributed file system



## Mounting

Mount mechanisms allow the binding together of different file namespaces to form a single hierarchical namespace. The Unix operating system uses this mechanism. A special entry, known as a mount point, is created at some position in the local file namespace that is bound to the root of another file name space. From the users point of view a mount point is indistinguishable from a local directory entry and may be traversed using standard path names once mounted. File access is therefore location transparent for the user but not for the system administrator. The kernel maintains a structure called a mount table which maps mount points to appropriate file systems. Whenever a file access path crosses a mount point, this is intercepted by the kernel, which then obtains the required service from the remote server.


Server A
a
b
g
h
i
Server B
d
e
f
Server C
j
k
l
Mount
Points

Client machines can maintain mount information individually as is done in Sun's Network File System. Every client individually mounts all the required file systems at specific points in its local file space. Note that each client need not necessarily mount the file systems at the same points. This makes it difficult to write portable code which can locate files in the distributed file system irrespective of where it is executed. Movement of files between servers requires each client to unmount and remount the affected subtree. Note that to simplify the implementation, Unix does not allow hard links to be placed across mount points, i.e. between two separate file systems. Also, servers are unaware of where the subtrees exported by them have been mounted.

Mount information can be maintained at servers in which case it is possible that every client sees an identical namespace. If files are moved to a different server then mount information need only be updated at the servers. Servers each manage domains or volumes which are subtrees of the overall name space. Clients may initially direct file requests to any server which will direct the client to the server which manages the domain containing the required file. This information may be used to guide the client more efficiently for future accesses.

## Client Caching

Caching is the architectural feature which contributes the most to performance in a distributed file system. Caching exploits temporal locality of reference. A copy of data stored at a remote server is brought to the client. Other metadata such as directories, protection and file status or location information also exhibit locality of reference and are good candidates for caching.

Data can be cached in main memory or on the local disk. A key issue is the size of cached units, whether entire files or individual file blocks are cached. Caching entire files is simpler and most files are in fact read sequentially in their entirety, but files which are larger than the client cache cannot be fetched. Cache validation can be done in two ways. The client can contact the server before accessing the cache or the server can notify clients when data is rendered stale. This can reduce client-server traffic.  A number of approaches to propagating changes back to the server are also possible. A change may be propagated when the file is closed or by deferring it for a set period of time. This would be useful where files are overwritten frequently. A policy of deferred update is also useful in situations where a host can become disconnected from the network and propagates updates upon reconnection.

## Hints

Caching is an important mechanism to improve the performance of a file system, however, guaranteeing the consistency of the cached items requires elaborate and expensive client/server protocols. An alternative approach to maintaining consistency is to treat cached data (mostly meta data) as hints. With this scheme, cached data is not expected to be completely consistent, but when it is, it can dramatically improve performance. For maximum performance benefit, a hint should nearly always be correct. To prevent the occurrence of negative consequences if a hint is erroneous, the classes of applications that use hints must be restricted to those that can recover after discovering that the cached data is invalid, that is, the data should be self validating upon use. Note that file data cannot be treated as a hint because use of the cached copy will not reveal whether it is current or stale.

For example, after the name of a file or directory is mapped to a physical object in the server, the address of that object can be stored as a hint in the client's cache. If the address fails to map to the object on a subsequent access, it is purged from the cache and the client must consult with the file server to determine the actual location of the object. It is unlikely that file locations will change frequently over time and so use of hints can benefit access performance.

# Bulk Data Transfer

All data transfer in a network requires the execution of various layers of communication protocols. Data is assembled and disassembled into packets, it is copied between the buffers of various layers in the communication protocols and transmitted in individually acknowledged packets over the network. For small amounts of data, the transit time across the network is low, but there are relatively high latency costs involved with the communication protocols establishing peer connections, packaging the small data packets for individual transmission and acknowledging receipt of each packet at each layer.

Transferring data in bulk reduces the relative cost of this overhead at the source and destination. With this scheme, multiple consecutive data packets are transferred from servers to clients (or vice versa) in a burst. At the source, multiple packets are formatted and transmitted with one context switch from client to kernel. At the destination, a single acknowledgement is used for the entire sequence of packets received.

# Encryption

Encryption is used for enforcing security in distributed systems. A number of possible threats exist such as unauthorised release of information, unauthorised modification of information, or unauthorised denial of resources. Encryption is primarily of value in preventing unauthorised release and modification of information.

The Kerberos protocol is most commonly used as a mechanism which employs encryption for initially establishing trusted communication between two parties and for generating a private session key for encrypting and decrypting subsequent messages between them. Private session keys tend to be shorter than those used for public key encryption and this makes it cheaper (easier) to perform the encryption.

For performance, encryption/decryption may be performed by special hardware at the client and server. It may be difficult to justify the cost or need for this hardware to users. The benefit is not as tangible as extra memory, processor speed or graphics capability and may be viewed as an expensive frill until the importance of security is perceived.

## Design Issues

1. Naming and Name Resolution:

Name refers to an object such as file or a directory. Name Resoltuion refers to the process of mapping a name to an object that is physical storage. Name space is collection of names. Names can be assigned to files in distributed file system in three ways:

a) Concatenate the host name to the names of files that are stored on that host.

Advantages:

File name is unique

System wide

Name resolution is simple as file can be located easily.

Limitations:

It conflicts with the goal of network transparency.

Moving a file from one host to another requires changes in filename and the application accessing that file that is naming scheme is not location independent.

b) Mount remote directories onto local directories. Mounting a remote directory require that host of directory to be known only once. Once a remote directory is mounted, its files can be referred in location transparent way. This approach resolve file name without consulting any host.

c) Maintain a single global directory where all the files in the system belong to single namespace. The main limitation of this scheme is that it is limited to one computing facility or to a few co-operating computing facilities. This scheme is not used generally.

Name Server: It is responsible for name resolution in distributed system. Generally two approaches are used for maintaining name resolution information.

2. Caches on Disk or Main Memory:

- Caching refers to storage of data either into the main memory or onto disk space after its first reference by client machine.

Advantages of having cache in main memory:

Disk less workstations can also take advantage of caching.

Accessing a cache in main memory is much faster than accessing a cache on local disk.

- The server cache is in the main memory at the server, a single design for a caching mechanism is used for clients and servers.

Limitations:

Large files cannot be cached completely so caching done block oriented which is more complex.

- It competes with virtual memory system for physical memory space, so a scheme to deal with memory contention cache and virtual memory system is necessary. Thus, more complex cache manager and memory management is required.

Advantages of having cache on a local disk:

Large files can be cached without affecting performance.

- Virtual memory management is simple.
- Portable workstation can be incorporated in distributed system.

## 3. Writing Policy:

- This policy decides when a modified cache block at client should be transferred to the server. Following policies are used:
- a) Write Through: All writes required by clients applications are also carried out at servers immediately. The main advantage is reliability that is when client crash, little information is lost. This scheme cannot take advantage of caching.
- b) Delayed Writing Policy: It delays the writing at the server that is modifications due to write are reflected at server after some delay. This scheme can take advantage of caching. The main limitation is less reliability that is when client crash, large amount of information can be lost.
- c) This scheme delays the updating of files at the server until the file is closed at the client. When average period for which files are open is short then this policy is equivalent to write through and when period is large then this policy is equivalent to delayed writing policy.

## 4. Cache Consistency:

- When multiple clients want to modify or access the same data, then cache consistency problem arises. Two schemes are used to guarantee that data returned to the client is valid.
- a) Server initiated approach: Server inform the cache manager whenever data in client caches becomes  valid. Cache manager at clients then retrieve the new data or invalidate the blocks containing old data in cache in cache. Server has maintain reliable records on what data blocks are cached by which cache managers. Co operation between servers and cache manager is required.
- b) Client-initiated approach: It is the responseabilty of cache manager at the clients to validate data with server before returning it to the clients. This approach does not take benefit of caching as the cache manager consult the server for validation of cached block each time.

## 5. Availability:

- It is one of the important issue is design of Distributed file system. Server failure or communication network can attract the availability of files.
- Replication: The primary mechanism used for enhancing availability of files is replication. In this mechanism, many copies or replicas of files are maintained at different servers.
- Limitations:

  Extra storage space is required to store replicas.

- Extra overhead is required in maintained all replicas up to date.

Following situations cause in consistency among replicas:

Replica is not updated due to failure of server storing the replica

- All the file servers storing the replicas of file are not reachable from all clients due to network partition and replicas of file in different partition are updated differently.

Unit of Replication: The main design issue in replication is the unit of replication. Following units can be used for replication of data:

- a) The most basic unit is file: This unit allow the replication of only those files that need to have higher availability. This unit results in expansive replica management. Protection rights associated with directory have to individually stored with each replica. Replica of common directory may not have common file servers so require extra name resolution to locate each replica in case of modification of file or directory.
- b) Group of files called volumes can be used: Volume may represent files of single user or files that are in a server. This scheme is used in coda file system. In this scheme, replica management is simple that is protection rights can be associated with volume instead with each individual file replica. Volume replication results in wasteful as a user needs higher availability for only a few files in the volume.
- c) Combination of volume and single file replication can be used: All the files of a user form a file group called primary pack. A replica of primary pack, called a pack, is allowed to contain a subset of files in primary pack. Corresponding to primary pack, one or more of packs can be obtained according to requirement.

## 6. Scalability:

- The design of DFS should be such that new systems can be easily introduced without affecting it. Generally, client-server organisation is used to define DFS structure. Caching is used in this organisation to improve performance. Server initiated cache invalidation is used to maintain cache consistency. In this approach, server maintain a record based information regarding all the clients sharing file stored on it. This information represents server state. As the system grows both the size of server state and load due to invalidations increases on server. Following schemes can be used to reduce server state and server load:
- a) Exploit knowledge about usage of files that is it is found that most commonly used and shared files are accessed in read only mode. So, there is no need to check the validity of these files of maintain the list of clients at servers for validation purpose.
- b) Generally, data required by a client is found in another client's cache so a client can obtain required data from another client rather than server.

Structure of server process play am important role. If server is designed with single process, then many clients have to wait for a long time whenever a disk input/ output is initiated. This can be avoided if separate process is assigned to each client.

7. Semantics:

The semantic of a file system represent the affects of accesses on file. The basic semantic is that a read operation will return the data (stored ) due to latest write operation. The semantic can be guaranteed in two ways: All read and writes from various clients will have to go through the server. Sharing will have to be disallowed either by server or by the use of locks by application. In first way, the server become bottleneck and in second way, the file is not available for certain clients.